



دانشگاه صنعتی امیرکبیر
دانشکده مهندسی کامپیوتر



راهنمای پروژه سامانه ثبت آگهی خرید و فروش دست دوم

آموزش مفاهیم پایه و راه اندازی بخش های پروژه

✂ راهنمای فنی پروژه

پروژه برنامه سازی پیشرفته

مسیر راه اندازی و کار با ابزارهای اصلی پروژه

JWT و HTTP

مفاهیم پایه

Frontend و Database، Backend

راه اندازی

GitHub و Git

مدیریت پروژه

اساتید

دکتر امیر کلباسی

دکتر معصومه طارمی راد

سرتدریسار

ابوالفضل دیباجی

پیاده سازی و تنظیم

سید حسام حسینیان

بهار ۱۴۰۵

فهرست مطالب

۱	آشنایی با درخواست‌های HTTP	۳
۱.۱	درخواست HTTP چیست؟	۳
۲.۱	متد GET	۳
۳.۱	متد POST	۴
۴.۱	متد PUT	۵
۵.۱	متد DELETE	۶
۶.۱	تفاوت متدهای اصلی HTTP	۶
۷.۱	ارسال داده در قالب JSON	۷
۸.۱	Status Code در پاسخ‌های HTTP	۷
۹.۱	نمونه پاسخ موفق	۸
۱۰.۱	نمونه پاسخ خطا	۸
۱۱.۱	نکته مهم برای این پروژه	۸
۲	آشنایی با احراز هویت با JWT	۹
۱.۲	JWT چیست؟	۹
۲.۲	چرا در این پروژه از JWT استفاده می‌شود؟	۹
۳.۲	ساختار کلی JWT	۹
۴.۲	جریان ورود کاربر با JWT	۱۰
۵.۲	ارسال JWT در درخواست‌های بعدی	۱۱
۶.۲	اعتبارسنجی JWT در Backend	۱۱
۷.۲	نگهداری JWT در Frontend	۱۲
۸.۲	خروج از حساب کاربری	۱۲
۹.۲	خطاهای رایج مربوط به JWT	۱۲
۱۰.۲	نکته مهم برای این پروژه	۱۳
۳	راهنمای راه‌اندازی Backend	۱۴
۱.۳	پیش‌نیازهای اجرای Backend	۱۴
۲.۳	ساخت پروژه با Maven و JetBrains IDE	۱۴
۳.۳	تنظیمات اولیه Backend	۱۵
۴.۳	اجرای Backend با Maven	۱۵
۵.۳	بررسی اجرای صحیح Backend	۱۶
۶.۳	آشنایی کوتاه با Postman	۱۶
۷.۳	خطاهای رایج هنگام اجرای Backend	۱۷

۱۸	۸.۳ خروجی مورد انتظار این مرحله
۱۸	۴ راهنمای راهاندازی Database
۱۸	۱.۴ گزینه اول: استفاده از PostgreSQL
۱۸	۱.۱.۴ اجرای PostgreSQL با Docker
۱۹	۲.۱.۴ اجرای PostgreSQL بدون Docker
۱۹	۳.۱.۴ تنظیم اتصال Backend به PostgreSQL
۲۰	۲.۴ گزینه دوم: استفاده از SQLite
۲۰	۱.۲.۴ افزودن وابستگی‌های SQLite
۲۱	۲.۲.۴ تنظیم اتصال Backend به SQLite
۲۱	۳.۲.۴ اجرای پروژه با SQLite
۲۱	۳.۴ استفاده از فایل‌های تنظیمات جداگانه
۲۲	۴.۴ نکات مهم
۲۲	۵.۴ خروجی مورد انتظار این مرحله
۲۳	۵ راهنمای راهاندازی Frontend با JavaFX
۲۳	۱.۵ پیش‌نیازهای اجرای Frontend
۲۳	۲.۵ افزودن وابستگی‌های JavaFX
۲۴	۳.۵ اجرای Frontend
۲۵	۴.۵ اتصال Frontend به Backend
۲۵	۵.۵ نمونه اجرای اولیه JavaFX
۲۶	۶.۵ نکات مهم در طراحی Frontend
۲۶	۷.۵ خطاهای رایج هنگام اجرای Frontend
۲۷	۸.۵ خروجی مورد انتظار این مرحله
۲۷	۶ راهنمای راهاندازی Repository در GitHub
۲۷	۱.۶ ساخت Repository در GitHub
۲۸	۲.۶ ساختار پیشنهادی Repository
۲۸	۳.۶ راهاندازی Git در سیستم
۲۹	۴.۶ دریافت پروژه از GitHub
۳۰	۵.۶ استفاده از commit در طول پروژه
۳۰	۶.۶ فایل .gitignore
۳۰	۷.۶ فایل README
۳۱	۸.۶ نحوه کار گروهی با Git
۳۲	۹.۶ مواردی که نباید در GitHub قرار گیرند
۳۲	۱۰.۶ تحویل Repository

آشنایی با درخواست‌های HTTP

در این پروژه، ارتباط بین Frontend و Backend از طریق درخواست‌های HTTP انجام می‌شود. Frontend که با JavaFX پیاده‌سازی می‌شود، برای انجام کارهایی مانند ثبت‌نام، ورود، دریافت لیست آگهی‌ها، ثبت آگهی جدید، ارسال پیام و امتیازدهی، به Backend درخواست ارسال می‌کند.

Backend نیز پس از دریافت درخواست، عملیات مورد نظر را انجام می‌دهد و نتیجه را در قالب یک پاسخ به Frontend برمی‌گرداند. معمولاً داده‌های ارسالی و دریافتی بین Frontend و Backend در قالب JSON هستند.

درخواست HTTP چیست؟

درخواست HTTP پیامی است که از سمت یک برنامه یا کاربر به یک سرور ارسال می‌شود. در این پروژه، Frontend نقش ارسال‌کننده درخواست را دارد و Backend نقش دریافت‌کننده و پردازش‌کننده درخواست را بر عهده دارد.

هر درخواست HTTP معمولاً شامل بخش‌های زیر است:

- متد درخواست، مانند GET یا POST

- آدرس API

- داده‌های ارسالی، در صورت نیاز

- Header در صورت نیاز

برای مثال، وقتی کاربر می‌خواهد لیست آگهی‌ها را مشاهده کند، Frontend یک درخواست به Backend ارسال می‌کند. Backend آگهی‌های فعال را از پایگاه داده دریافت می‌کند و نتیجه را به Frontend برمی‌گرداند.

متد GET

متد GET برای دریافت اطلاعات از Backend استفاده می‌شود. این متد نباید باعث ایجاد، ویرایش یا حذف داده در پایگاه داده شود.

نمونه کاربردهای GET در این پروژه:

- دریافت لیست آگهی‌های فعال

- دریافت جزئیات یک آگهی

- دریافت لیست دسته‌بندی‌ها

- دریافت لیست شهرها

- دریافت پیام‌های یک گفت‌وگو

● دریافت لیست علاقه‌مندی‌های کاربر

نمونه مسیر:

GET /api/advertisements

این درخواست از Backend می‌خواهد لیست آگهی‌های فعال را برگرداند.

نمونه دیگر:

GET /api/advertisements/5

این درخواست جزئیات آگهی با شناسه ۵ را دریافت می‌کند.

متد POST

متد POST برای ایجاد داده جدید در سیستم استفاده می‌شود. هر زمانی که قرار است یک رکورد جدید در پایگاه داده ساخته شود، معمولاً از POST استفاده می‌شود.

نمونه کاربردهای POST در این پروژه:

- ثبت نام کاربر جدید
- ورود کاربر
- ثبت آگهی جدید
- ارسال پیام جدید
- ثبت امتیاز برای فروشنده
- افزودن آگهی به علاقه‌مندی‌ها

نمونه مسیر:

POST /api/auth/register

نمونه داده ارسالی برای ثبت نام:

```
{
  "fullName": "Hessam Hosseini",
  "username": "wasHessam",
  "password": "123456",
  "phoneNumber": "09123456789"
}
```

در این حالت، Frontend اطلاعات کاربر را به Backend ارسال می‌کند و Backend یک کاربر جدید در پایگاه داده ایجاد می‌کند.

نمونه دیگر:

POST /api/advertisements

این درخواست برای ثبت آگهی جدید استفاده می‌شود.

متد PUT

متد PUT برای ویرایش یا به‌روزرسانی اطلاعات موجود استفاده می‌شود. زمانی که یک داده از قبل در سیستم وجود دارد و می‌خواهیم آن را تغییر دهیم، معمولاً از PUT استفاده می‌کنیم.

نمونه کاربردهای PUT در این پروژه:

- ویرایش اطلاعات کاربر
- ویرایش آگهی
- تغییر وضعیت آگهی به فروخته‌شده
- تایید آگهی توسط مدیر
- رد کردن آگهی توسط مدیر
- ویرایش دسته‌بندی
- مسدود کردن یا فعال‌سازی کاربر

نمونه مسیر:

PUT /api/advertisements/5

این درخواست برای ویرایش آگهی با شناسه ۵ استفاده می‌شود.

نمونه داده ارسالی:

```
{
  "title": "Updated title",
  "description": "Updated description",
  "price": 25000000
}
```

نمونه دیگر:

PUT /api/admin/advertisements/5/approve

این درخواست توسط مدیر برای تایید آگهی با شناسه ۵ استفاده می‌شود.

متد DELETE

متد DELETE برای حذف داده استفاده می‌شود. در این پروژه، حذف می‌تواند به دو شکل انجام شود:

- حذف واقعی از پایگاه داده
 - حذف منطقی با تغییر وضعیت داده
- در بسیاری از سیستم‌ها، به جای حذف کامل آگهی از پایگاه داده، وضعیت آن به حذف‌شده تغییر داده می‌شود. این روش بهتر است، زیرا سابقه اطلاعات در سیستم باقی می‌ماند.
- نمونه کاربردهای DELETE در این پروژه:

- حذف آگهی توسط صاحب آگهی
- حذف آگهی نامناسب توسط مدیر
- حذف آگهی از علاقه‌مندی‌ها
- حذف یا غیرفعال کردن دسته‌بندی

نمونه مسیر:

DELETE /api/advertisements/5

این درخواست برای حذف آگهی با شناسه ۵ استفاده می‌شود.

نمونه دیگر:

DELETE /api/favorites/10

این درخواست یک آگهی را از لیست علاقه‌مندی‌های کاربر حذف می‌کند.

تفاوت متدهای اصلی HTTP

به صورت خلاصه، کاربرد متدهای اصلی به شکل زیر است:

کاربرد	متد
دریافت اطلاعات از Backend، بدون تغییر دادن داده‌ها	GET
ایجاد داده جدید، مانند ثبت نام، ثبت آگهی یا ارسال پیام	POST
ویرایش یا به‌روزرسانی داده موجود	PUT
حذف داده یا تغییر وضعیت آن به حذف شده	DELETE

ارسال داده در قالب JSON

در بیشتر API‌های این پروژه، داده‌ها در قالب JSON بین Frontend و Backend رد و بدل می‌شوند. JSON یک قالب متنی ساده برای نمایش داده‌ها است و به راحتی در Java قابل پردازش است. نمونه JSON مربوط به ثبت آگهی:

```
{
  "title": "Laptop Lenovo",
  "description": "Used laptop in good condition",
  "price": 180000000,
  "categoryId": 2,
  "cityId": 1,
  "ownerId": 3
}
```

در این مثال، Frontend اطلاعات آگهی را به Backend ارسال می‌کند. Backend این داده‌ها را دریافت کرده، اعتبارسنجی می‌کند و در صورت معتبر بودن، آگهی را در پایگاه داده ذخیره می‌کند.

Status Code در پاسخ‌های HTTP

Backend علاوه بر داده، معمولاً یک کد وضعیت نیز برمی‌گرداند. این کد نشان می‌دهد درخواست موفق بوده یا با خطا مواجه شده است.

کدهای رایج در این پروژه:

● 200: درخواست با موفقیت انجام شده است.

● 201: داده جدید با موفقیت ساخته شده است.

● 400: داده ارسالی نامعتبر است.

● 401: کاربر وارد سیستم نشده است.

● 403: کاربر اجازه انجام این عملیات را ندارد.

● 404: داده مورد نظر پیدا نشده است.

● 500: خطای داخلی در Backend رخ داده است.

برای مثال، اگر کاربر بخواهد آگهی‌ای را مشاهده کند که وجود ندارد، Backend می‌تواند پاسخ 404 برگرداند.

📌 نمونه پاسخ موفق

نمونه پاسخ موفق برای دریافت جزئیات آگهی:

```
{
  "id": 5,
  "title": "Laptop Lenovo",
  "description": "Used laptop in good condition",
  "price": 180000000,
  "city": "Tehran",
  "category": "Digital Goods",
  "status": "ACTIVE"
}
```

📌 نمونه پاسخ خطا

نمونه پاسخ خطا:

```
{
  "message": "Advertisement not found",
  "status": 404
}
```

Frontend باید این خطا را دریافت کند و پیام مناسبی به کاربر نمایش دهد.

📌 نکته مهم برای این پروژه

در این پروژه، Frontend نباید مستقیماً به پایگاه داده متصل شود. تمام عملیات باید از طریق API‌های Backend انجام شود.

رعایت این ساختار باعث می‌شود پروژه منظم‌تر، قابل توسعه‌تر و نزدیک‌تر به معماری پروژه‌های واقعی باشد.

📖 آشنایی با احراز هویت با JWT

در پروژه‌هایی که Frontend و Backend از طریق HTTP با هم ارتباط دارند، معمولاً لازم است Backend بدانند هر درخواست از طرف کدام کاربر ارسال شده است. برای مثال، وقتی کاربر می‌خواهد آگهی جدید ثبت کند، پیام ارسال کند، آگهی را به علاقه‌مندی‌ها اضافه کند یا وارد پنل مدیریت شود، سیستم باید ابتدا هویت و سطح دسترسی او را بررسی کند.

یکی از روش‌های رایج برای این کار، استفاده از JWT است. JWT مخفف JSON Web Token است و به زبان ساده، یک رشته متنی امضا شده است که اطلاعات لازم درباره کاربر را در خود نگه می‌دارد. پس از ورود موفق کاربر، Backend یک JWT تولید می‌کند و آن را به Frontend برمی‌گرداند. از آن به بعد، Frontend این توکن را همراه درخواست‌های بعدی به Backend ارسال می‌کند.

JWT چیست؟

JWT یک قالب استاندارد برای انتقال امن اطلاعات بین دو بخش از سیستم است. این اطلاعات معمولاً شامل شناسه کاربر، نام کاربری، نقش کاربر و زمان انقضای توکن است.

برای مثال، در این پروژه، JWT می‌تواند مشخص کند که کاربر وارد شده چه شناسه‌ای دارد و نقش او کاربر عادی است یا مدیر سیستم. Backend با خواندن و اعتبارسنجی توکن، می‌تواند تصمیم بگیرد که کاربر اجازه انجام یک عملیات خاص را دارد یا نه.

چرا در این پروژه از JWT استفاده می‌شود؟

در این پروژه، بسیاری از قابلیت‌ها وابسته به کاربر وارد شده هستند. چند نمونه:

- ثبت آگهی باید به حساب کاربر وارد شده متصل شود.
- کاربر فقط باید بتواند آگهی‌های خودش را ویرایش یا حذف کند.
- ارسال پیام باید با هویت کاربر انجام شود.
- علاقه‌مندی‌ها باید برای هر کاربر جداگانه ذخیره شوند.
- فقط مدیر سیستم باید به پنل مدیریت دسترسی داشته باشد.

بنابراین Backend نباید فقط به داده‌هایی که Frontend ارسال می‌کند اعتماد کند. برای مثال، اگر Frontend یک ownerId ارسال کند، Backend باید بررسی کند که این شناسه واقعاً متعلق به کاربر وارد شده است. JWT به Backend کمک می‌کند هویت کاربر را از روی توکن معتبر تشخیص دهد.

ساختار کلی JWT

یک JWT معمولاً از سه بخش تشکیل می‌شود که با نقطه از هم جدا شده‌اند:

header.payload.signature

بخش Header مشخص می‌کند نوع توکن چیست و از چه الگوریتمی برای امضا استفاده شده است. بخش Payload شامل اطلاعات اصلی مانند شناسه کاربر، نام کاربری، نقش و زمان انقضا است. بخش Signature نیز برای اطمینان از تغییر نکردن توکن استفاده می‌شود.

نمونه ظاهری یک توکن:

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbGkiLCJyb2xlIjoiiVVNFUiJ9.signature
```

دانشجو لازم نیست این رشته را دستی تولید کند یا تغییر دهد. تولید و بررسی JWT باید توسط کتابخانه‌های معتبر در Backend انجام شود.

جریان ورود کاربر با JWT

جریان معمول ورود کاربر در این پروژه به شکل زیر است:

۱. کاربر در Frontend نام کاربری و رمز عبور را وارد می‌کند.
۲. Frontend درخواست ورود را به Backend ارسال می‌کند.
۳. Backend نام کاربری و رمز عبور را بررسی می‌کند.
۴. اگر اطلاعات درست باشد، Backend یک JWT تولید می‌کند.
۵. Backend توکن را در پاسخ ورود به Frontend برمی‌گرداند.
۶. Frontend توکن را نگهداری می‌کند.
۷. در درخواست‌های بعدی، Frontend توکن را همراه درخواست ارسال می‌کند.

نمونه درخواست ورود:

POST /api/auth/login

```
{
  "username": "wasHessam",
  "password": "123456"
}
```

نمونه پاسخ موفق:

```
{
  "token": "jwt-token-here",
  "userId": 3,
  "username": "wasHessam",
  "role": "USER"
}
```

ارسال JWT در درخواست‌های بعدی

پس از ورود موفق، Frontend باید توکن را در درخواست‌هایی که نیاز به ورود دارند ارسال کند. روش رایج، قرار دادن توکن در Header درخواست با نام Authorization است.

فرمت معمول به شکل زیر است:

```
Authorization: Bearer jwt-token-here
```

برای مثال، اگر کاربر بخواهد آگهی جدید ثبت کند، درخواست می‌تواند به شکل زیر باشد:

```
POST /api/advertisements
Authorization: Bearer jwt-token-here
```

```
{
  "title": "Laptop Lenovo",
  "description": "Used laptop in good condition",
  "price": 180000000,
  "categoryId": 2,
  "cityId": 1
}
```

در این حالت، بهتر است Backend شناسه صاحب آگهی را از روی JWT تشخیص دهد، نه اینکه به ownerId ارسالی از سمت Frontend اعتماد کند.

اعتبارسنجی JWT در Backend

در هر درخواست محافظت‌شده، Backend باید توکن را بررسی کند. موارد مهم در اعتبارسنجی عبارت‌اند از:

- توکن وجود داشته باشد.

- فرمت توکن درست باشد.

- امضای توکن معتبر باشد.
- توکن منقضی نشده باشد.
- کاربر مربوط به توکن هنوز در سیستم وجود داشته باشد.
- کاربر مسدود نشده باشد.
- نقش کاربر برای عملیات مورد نظر کافی باشد.

برای مثال، اگر یک کاربر عادی بخواهد آگهی‌ها را تایید یا رد کند، حتی اگر توکن معتبر داشته باشد، Backend باید درخواست او را رد کند؛ چون این عملیات فقط برای مدیر سیستم مجاز است.

نگهداری JWT در Frontend

در Frontend مبتنی بر JavaFX، پس از ورود موفق، توکن می‌تواند در یک کلاس کمکی مانند SessionManager نگهداری شود. این کلاس می‌تواند اطلاعات کاربر واردشده و توکن فعلی را تا زمان خروج کاربر نگه دارد. سپس کلاس‌های مربوط به ارسال درخواست به Backend می‌توانند هنگام ساخت درخواست، مقدار توکن را از SessionManager دریافت کنند و در Header قرار دهند.

خروج از حساب کاربری

در ساده‌ترین حالت، خروج از حساب کاربری در Frontend یعنی حذف کردن توکن ذخیره‌شده. وقتی توکن از SessionManager حذف شود، Frontend دیگر نمی‌تواند درخواست‌های نیازمند ورود را با هویت کاربر قبلی ارسال کند.

اگر پروژه بخواهد پیشرفته‌تر باشد، می‌توان در Backend نیز لیستی از توکن‌های باطل‌شده نگهداری کرد، اما برای این پروژه آموزشی، حذف توکن از سمت Frontend معمولاً کافی است.

خطاهای رایج مربوط به JWT

در زمان کار با JWT ممکن است خطاهای مختلفی رخ دهد:

- ارسال نشدن توکن در درخواست
- اشتباه بودن فرمت Authorization
- منقضی شدن توکن
- نامعتبر بودن امضای توکن
- ارسال توکن مربوط به کاربر مسدودشده
- تلاش کاربر عادی برای دسترسی به API‌های مدیر

در این حالت‌ها، Backend باید پاسخ مناسب برگرداند. برای مثال، اگر توکن ارسال نشده یا نامعتبر باشد، پاسخ 401 مناسب است. اگر توکن معتبر باشد اما کاربر اجازه انجام عملیات را نداشته باشد، پاسخ 403 مناسب‌تر است.

نکته مهم برای این پروژه

JWT نباید به عنوان تنها معیار اعتماد به داده‌های ورودی استفاده شود. داشتن توکن معتبر یعنی کاربر وارد سیستم شده است، اما همچنان باید قوانین منطقی پروژه بررسی شوند. برای مثال، کاربر وارد شده نباید بتواند آگهی کاربر دیگر را ویرایش کند، به آگهی خودش پیام بدهد یا بدون نقش مدیر به پنل مدیریت دسترسی داشته باشد.

در این پروژه، مسیرهای عمومی مانند ثبت‌نام، ورود، مشاهده لیست آگهی‌ها و مشاهده جزئیات آگهی می‌توانند بدون توکن قابل دسترسی باشند. اما مسیرهایی مانند ثبت آگهی، ویرایش آگهی، ارسال پیام، علاقه‌مندی‌ها، امتیازدهی و پنل مدیریت باید با JWT محافظت شوند.

راهنمای راه اندازی Backend

در این بخش، نحوه راه اندازی بخش Backend پروژه توضیح داده می شود. Backend سامانه مسئول پیاده سازی منطق اصلی برنامه، مدیریت کاربران، ثبت و مدیریت آگهی ها، ارتباط با پایگاه داده و ارائه API های مورد نیاز Frontend است.

در این پروژه، Backend با زبان Java و فریم ورک Spring Boot پیاده سازی می شود. دانشجویان باید ابتدا پروژه Backend را اجرا کنند و مطمئن شوند که سرور بدون خطا بالا می آید.

پیش نیازهای اجرای Backend

برای اجرای Backend، نصب بودن ابزارهای زیر روی سیستم الزامی است:

● Java Development Kit نسخه ۱۷ یا بالاتر

● Maven

● یک محیط توسعه مانند IntelliJ IDEA یا VS Code

● دسترسی به ترمینال یا Command Prompt

برای بررسی نصب بودن Java و Maven، می توان از دستورهای زیر استفاده کرد:

```
java -version
```

```
mvn -version
```

در صورتی که نسخه Java یا Maven نمایش داده نشود، ابتدا باید ابزار مربوطه نصب و مسیر آن در متغیرهای محیطی سیستم تنظیم شود.

ساخت پروژه Maven با JetBrains IDE

دانشجویان می توانند پروژه Backend را به صورت مستقیم با استفاده از IDE های JetBrains، مانند IntelliJ IDEA، ایجاد کنند. برای این کار کافی است هنگام ساخت پروژه جدید، نوع پروژه Maven یا Spring Boot انتخاب شود.

در صورت استفاده از Spring Boot، می توان وابستگی های مورد نیاز مانند Spring Web، Spring Data JPA و درایور پایگاه داده را هنگام ساخت پروژه انتخاب کرد. پس از ایجاد پروژه، فایل pom.xml به صورت خودکار ساخته می شود و دانشجو می تواند ساختار لایه ای پروژه را در پوشه src/main/java ایجاد کند.

در صورتی که پروژه از قبل ساخته شده باشد، می توان آن را از طریق گزینه Open در IntelliJ IDEA باز کرد. IDE فایل pom.xml را تشخیص می دهد و وابستگی های Maven را به صورت خودکار بارگذاری می کند.

برای اجرای پروژه از داخل IntelliJ IDEA نیز می‌توان کلاس `SecondHandApplication.java` را اجرا کرد یا از ترمینال داخلی IDE دستور زیر را وارد نمود:

```
mvn spring-boot:run
```

تنظیمات اولیه Backend

تنظیمات اصلی Backend در فایل زیر قرار می‌گیرد:

```
src/main/resources/application.properties
```

در این فایل، مواردی مانند پورت اجرای سرور، مشخصات اتصال به پایگاه داده و تنظیمات مربوط به JPA مشخص می‌شوند. برای اجرای اولیه پروژه، می‌توان از تنظیمات نمونه زیر استفاده کرد:

```
server.port=8080
```

```
spring.application.name=secondhand-backend
```

با این تنظیمات، Backend روی پورت ۸۰۸۰ اجرا می‌شود و آدرس اصلی آن به صورت زیر خواهد بود:

```
http://localhost:8080
```

اجرای Backend با Maven

برای اجرای Backend، ابتدا وارد پوشه Backend شوید:

```
cd backend
```

سپس دستور زیر را اجرا کنید:

```
mvn spring-boot:run
```

اگر همه تنظیمات درست باشد، برنامه بدون خطا اجرا می‌شود و در خروجی ترمینال پیامی مشابه اجرای موفق Spring Boot نمایش داده می‌شود. پس از بالا آمدن سرور، Backend آماده دریافت درخواست از سمت Frontend خواهد بود.

بررسی اجرای صحیح Backend

برای بررسی اینکه Backend به درستی اجرا شده است، می‌توان یک API ساده برای تست سلامت سرور تعریف کرد. برای مثال، یک Controller با نام HealthController ساخته می‌شود که در صورت اجرای صحیح برنامه، یک پیام ساده برمی‌گرداند.

```
@RestController
@RequestMapping("/api/health")
public class HealthController {

    @GetMapping
    public String checkHealth() {
        return "Backend is running";
    }
}
```

پس از اجرای برنامه، آدرس زیر را در مرورگر باز کنید:

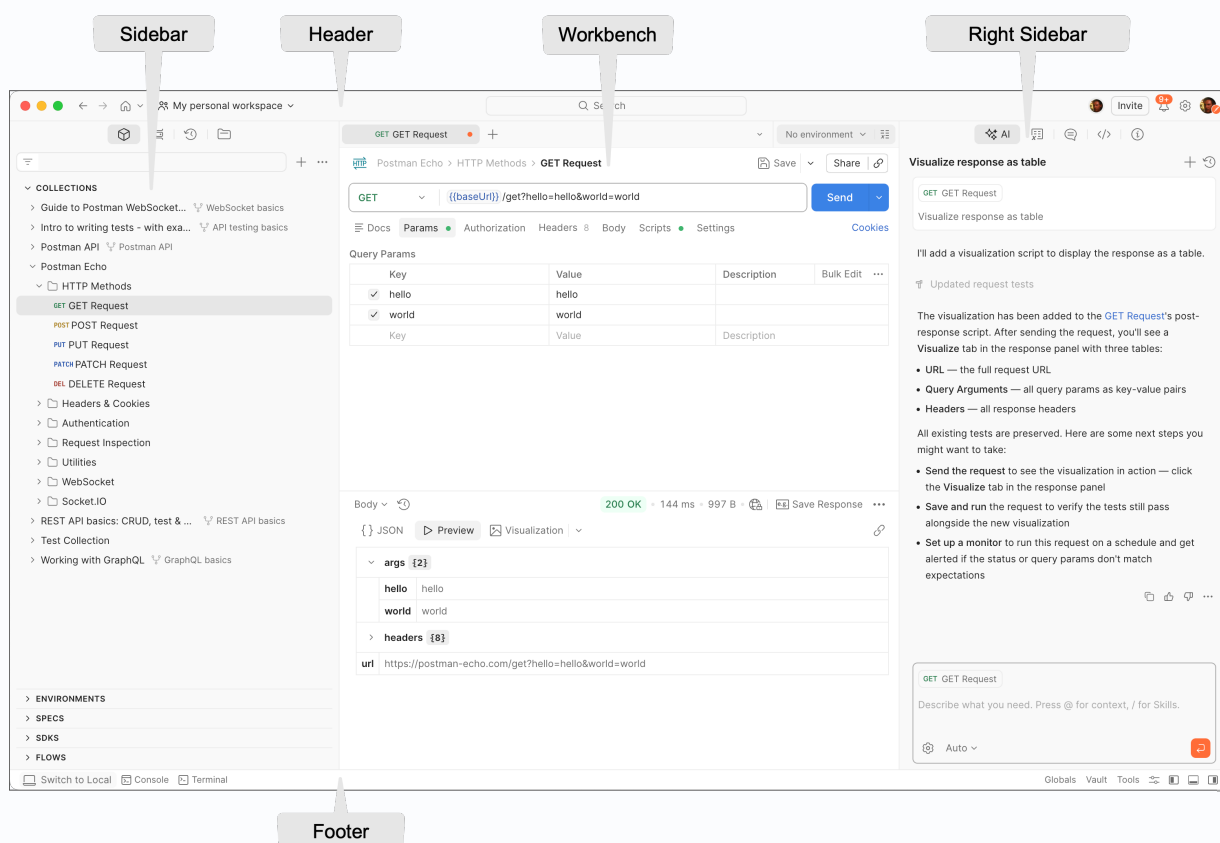
`http://localhost:8080/api/health`

اگر پیام زیر نمایش داده شود، یعنی Backend با موفقیت اجرا شده است:

Backend is running

آشنایی کوتاه با Postman

Postman ابزاری برای تست و بررسی API‌ها است. با استفاده از آن می‌توان بدون نیاز به آماده بودن Frontend، درخواست‌هایی مانند GET، POST، PUT و DELETE را به Backend ارسال کرد، داده‌های JSON را در بدنه درخواست قرار داد، توکن JWT یا سایر Headerها را تنظیم کرد و پاسخ سرور را همراه با کد وضعیت مشاهده نمود. این ابزار برای تست مسیرهایی مانند ثبت‌نام، ورود، ثبت آگهی، ویرایش آگهی و بررسی خطاهای Backend بسیار کاربردی است.



شکل ۱: نمونه‌ای از محیط برنامه Postman

خطاهای رایج هنگام اجرای Backend

در هنگام اجرای Backend ممکن است خطاهای مختلفی رخ دهد. چند مورد از خطاهای رایج عبارت‌اند از:

- نصب نبودن Java یا تنظیم نبودن مسیر آن

- نصب نبودن Maven

- اشغال بودن پورت ۸۰۸۰ توسط برنامه‌ای دیگر

- وجود خطا در فایل `pom.xml`

- ناقص بودن Dependency‌های پروژه

- اشتباه بودن تنظیمات اتصال به پایگاه داده

اگر پورت ۸۰۸۰ اشغال باشد، می‌توان پورت اجرای برنامه را در فایل `application.properties` تغییر داد:

```
server.port=8081
```

در این حالت، آدرس Backend به شکل زیر تغییر می‌کند:

http://localhost:8081

خروجی مورد انتظار این مرحله

در پایان این مرحله، دانشجو باید بتواند Backend پروژه را اجرا کند و از طریق مرورگر یا ابزارهایی مانند Postman یک درخواست ساده به سرور ارسال کند. اجرای موفق مسیر تست سلامت سرور نشان می‌دهد که Backend به درستی بالا آمده و آماده توسعه بخش‌های اصلی پروژه است.

راهنمای راه‌اندازی Database

در این پروژه، اطلاعات سامانه مانند کاربران، آگهی‌ها، دسته‌بندی‌ها، پیام‌ها، گزارش‌ها و علاقه‌مندی‌ها باید به صورت پایدار ذخیره شوند تا پس از بستن و اجرای مجدد سیستم از بین نروند. این ذخیره‌سازی می‌تواند با استفاده از پایگاه داده یا با استفاده از فایل انجام شود.

استفاده از پایگاه داده در این پروژه اجباری نیست. دانشجویان می‌توانند برای ساده‌تر شدن پیاده‌سازی، اطلاعات برنامه را در فایل‌هایی مانند JSON، CSV یا فایل متنی ذخیره کنند، به شرطی که داده‌ها پس از اجرای مجدد برنامه قابل بازیابی باشند و منطق اصلی سیستم درست کار کند.

پایگاه داده پیشنهادی برای این پروژه PostgreSQL است. با این حال، دانشجویان می‌توانند از پایگاه داده‌های رابطه‌ای مشابه مانند MySQL، MariaDB یا H2 نیز استفاده کنند، به شرطی که ساختار جداول، ارتباط بین موجودیت‌ها و نحوه ذخیره‌سازی داده‌ها به درستی پیاده‌سازی شود.

برای ساده‌تر شدن اجرای پروژه، استفاده از SQLite نیز مجاز است. SQLite نیاز به راه‌اندازی سرور جداگانه ندارد و اطلاعات را داخل یک فایل ذخیره می‌کند. بنابراین برای پروژه‌های کوچک و آموزشی گزینه مناسبی است.

گزینه اول: استفاده از PostgreSQL

PostgreSQL یک پایگاه داده رابطه‌ای قدرتمند است که برای پروژه‌های واقعی و بزرگ نیز استفاده می‌شود. در این پروژه، استفاده از PostgreSQL به عنوان گزینه اصلی پیشنهاد می‌شود.

اجرای PostgreSQL با Docker

ساده‌ترین روش برای راه‌اندازی PostgreSQL استفاده از Docker است. ابتدا مطمئن شوید Docker روی سیستم نصب و اجرا شده است. سپس دستور زیر را اجرا کنید:

```
docker run --name secondhand-postgres \
  -e POSTGRES_USER=secondhand_user \
  -e POSTGRES_PASSWORD=secondhand_password \
  -e POSTGRES_DB=secondhand_db \
```

```
-p 5432:5432 \  
-d postgres
```

در این دستور، یک پایگاه داده با نام `secondhand_db` ساخته می‌شود. نام کاربری پایگاه داده `secondhand_user` و رمز عبور آن `secondhand_password` است. پایگاه داده روی پورت 5432 اجرا می‌شود. برای بررسی اینکه کانتینر PostgreSQL به درستی اجرا شده است، می‌توان از دستور زیر استفاده کرد:

```
docker ps
```

اگر کانتینر `secondhand-postgres` در لیست نمایش داده شود، یعنی پایگاه داده با موفقیت بالا آمده است.

در صورتی که کانتینر متوقف شده باشد، می‌توان آن را با دستور زیر دوباره اجرا کرد:

```
docker start secondhand-postgres
```

برای متوقف کردن پایگاه داده نیز از دستور زیر استفاده می‌شود:

```
docker stop secondhand-postgres
```

◀ اجرای PostgreSQL بدون Docker

در صورتی که دانشجو از Docker استفاده نکند، می‌تواند PostgreSQL را به صورت مستقیم روی سیستم نصب کند. پس از نصب، باید یک پایگاه داده برای پروژه ساخته شود. نمونه دستورات ساخت پایگاه داده و کاربر به صورت زیر است:

```
CREATE DATABASE secondhand_db;  
CREATE USER secondhand_user WITH PASSWORD 'secondhand_password';  
GRANT ALL PRIVILEGES ON DATABASE secondhand_db TO secondhand_user;
```

این دستورات را می‌توان از طریق ابزارهایی مانند `psql` یا نرم‌افزارهای مدیریت پایگاه داده مانند `pgAdmin` اجرا کرد.

◀ تنظیم اتصال Backend به PostgreSQL

برای اتصال Backend به PostgreSQL، باید تنظیمات زیر در فایل `application.properties` قرار داده شود:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/secondhand_db
spring.datasource.username=secondhand_user
spring.datasource.password=secondhand_password
spring.datasource.driver-class-name=org.postgresql.Driver
```

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
```

گزینه `spring.jpa.hibernate.ddl-auto=update` باعث می‌شود Hibernate بر اساس کلاس‌های Entity، ساختار جداول را تا حد امکان به صورت خودکار ایجاد یا به‌روزرسانی کند. استفاده از این گزینه برای محیط آموزشی مناسب است، اما در پروژه‌های واقعی باید با دقت بیشتری انجام شود. همچنین باید وابستگی PostgreSQL در فایل `pom.xml` وجود داشته باشد:

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

در صورتی که از JPA استفاده می‌شود، وابستگی زیر نیز باید در پروژه وجود داشته باشد:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

گزینه دوم: استفاده از SQLite

SQLite یک پایگاه داده سبک و فایل‌محور است. برخلاف PostgreSQL، برای استفاده از SQLite نیازی به اجرای سرور جداگانه نیست. اطلاعات برنامه داخل یک فایل ذخیره می‌شود. به همین دلیل، SQLite برای پروژه‌های آموزشی و اجرای ساده‌تر گزینه مناسبی است.

در این حالت، کافی است Backend اجرا شود و فایل پایگاه داده به صورت خودکار ساخته شود.

افزودن وابستگی‌های SQLite

برای استفاده از SQLite در Spring Boot، باید وابستگی‌های مربوط به SQLite و Dialect آن در فایل `pom.xml` اضافه شود:

```
<dependency>
  <groupId>org.xerial</groupId>
  <artifactId>sqlite-jdbc</artifactId>
</dependency>
```

```
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-community-dialects</artifactId>
</dependency>
```

در بعضی نسخه‌ها ممکن است لازم باشد برای وابستگی sqlite-jdbc یک نسخه مشخص نیز تعیین شود.

◀ تنظیم اتصال Backend به SQLite

برای استفاده از SQLite، تنظیمات زیر را در فایل application.properties قرار دهید:

```
spring.datasource.url=jdbc:sqlite:secondhand.db
spring.datasource.driver-class-name=org.sqlite.JDBC

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.database-platform=org.hibernate.community.dialect.SQLiteDialect
```

با این تنظیمات، هنگام اجرای Backend، فایلی با نام secondhand.db در مسیر اجرای پروژه ساخته می‌شود. این فایل شامل داده‌های پایگاه داده خواهد بود.

◀ اجرای پروژه با SQLite

پس از اضافه کردن وابستگی‌ها و تنظیمات، کافی است Backend را اجرا کنید:

```
mvn spring-boot:run
```

اگر تنظیمات درست باشد، برنامه بدون نیاز به اجرای پایگاه داده جداگانه بالا می‌آید و فایل secondhand.db ساخته می‌شود.

📌 استفاده از فایل‌های تنظیمات جداگانه

برای اینکه دانشجو بتواند بین PostgreSQL و SQLite جابه‌جا شود، بهتر است برای هر پایگاه داده یک فایل تنظیمات جداگانه ساخته شود.

برای PostgreSQL:

```
src/main/resources/application-postgres.properties
```

برای SQLite:

```
src/main/resources/application-sqlite.properties
```

سپس می‌توان هنگام اجرای پروژه، Profile موردنظر را مشخص کرد.

اجرای پروژه با PostgreSQL:

```
mvn spring-boot:run -Dspring-boot.run.profiles=postgres
```

اجرای پروژه با SQLite:

```
mvn spring-boot:run -Dspring-boot.run.profiles=sqlite
```

در این روش، دانشجو می‌تواند بدون تغییر دادن مداوم فایل اصلی تنظیمات، پایگاه داده موردنظر خود را انتخاب کند.

نکات مهم

در هر بار اجرای پروژه، فقط باید یکی از روش‌های ذخیره‌سازی فعال باشد. یعنی دانشجو باید تصمیم بگیرد که پروژه را با PostgreSQL، یا با SQLite یا با ذخیره‌سازی فایل‌محور اجرا می‌کند.

استفاده از PostgreSQL به عنوان گزینه اصلی پیشنهاد می‌شود، زیرا به ساختار پروژه‌های واقعی نزدیک‌تر است. با این حال، SQLite یا ذخیره‌سازی در فایل برای دانشجویانی که می‌خواهند پروژه را سریع‌تر و ساده‌تر اجرا کنند، قابل قبول است.

همچنین دانشجویان باید دقت کنند که اطلاعات حساس مانند نام کاربری و رمز عبور پایگاه داده نباید در پروژه‌های واقعی به صورت مستقیم داخل کد قرار گیرد. اما برای این پروژه آموزشی، استفاده از مقادیر ساده و مشخص در فایل تنظیمات قابل قبول است.

خروجی مورد انتظار این مرحله

در پایان این مرحله، دانشجو باید بتواند یکی از روش‌های ذخیره‌سازی، مانند PostgreSQL، SQLite یا فایل را برای پروژه تنظیم کند. پس از اجرای Backend، داده‌های اصلی پروژه باید به صورت پایدار ذخیره و بازیابی شوند و برنامه بدون خطای مربوط به ذخیره‌سازی اجرا شود.

راهنمای راهاندازی Frontend با JavaFX

در این پروژه، بخش Frontend باید با استفاده از JavaFX پیاده‌سازی شود. JavaFX یک کتابخانه گرافیکی برای زبان Java است که امکان ساخت رابط کاربری دسکتاپ را فراهم می‌کند. در این پروژه، Frontend وظیفه نمایش صفحه‌های مختلف سامانه، دریافت ورودی از کاربر و ارسال درخواست به Backend را بر عهده دارد. رابط کاربری پروژه باید به گونه‌ای طراحی شود که کاربر بتواند عملیات اصلی سامانه، مانند ثبت‌نام، ورود، مشاهده آگهی‌ها، ثبت آگهی جدید، جست‌وجو، فیلتر، مشاهده جزئیات آگهی و ارسال پیام را از طریق پنجره‌های گرافیکی انجام دهد.

پیش‌نیازهای اجرای Frontend

برای اجرای Frontend با JavaFX، نصب بودن ابزارهای زیر روی سیستم الزامی است:

● Java Development Kit نسخه ۱۷ یا بالاتر

● Maven

● یک محیط توسعه مانند IntelliJ IDEA، Eclipse یا VS Code

● دسترسی به ترمینال یا Command Prompt

برای بررسی نصب بودن Java و Maven، می‌توان از دستورهای زیر استفاده کرد:

```
java -version
```

```
mvn -version
```

در صورتی که نسخه Java یا Maven نمایش داده نشود، ابتدا باید ابزار مربوطه نصب و مسیر آن در متغیرهای محیطی سیستم تنظیم شود.

افزودن وابستگی‌های JavaFX

برای استفاده از JavaFX در پروژه Maven، باید وابستگی‌های مربوط به JavaFX به فایل pom.xml اضافه شوند. نمونه‌ای از تنظیمات موردنیاز به صورت زیر است:

```
<dependencies>
  <dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-controls</artifactId>
    <version>21.0.2</version>
```

```

</dependency>

<dependency>
  <groupId>org.openjfx</groupId>
  <artifactId>javafx-fxml</artifactId>
  <version>21.0.2</version>
</dependency>
</dependencies>

```

همچنین برای اجرای ساده‌تر برنامه، می‌توان از پلاگین JavaFX Maven استفاده کرد:

```

<build>
  <plugins>
    <plugin>
      <groupId>org.openjfx</groupId>
      <artifactId>javafx-maven-plugin</artifactId>
      <version>0.0.8</version>
      <configuration>
        <mainClass>com.example.secondhandfx.MainApplication</mainClass>
      </configuration>
    </plugin>
  </plugins>
</build>

```

نام کلاس اصلی باید با مسیر واقعی کلاس اجرای برنامه در پروژه دانشجویی هماهنگ باشد.

اجرای Frontend

برای اجرای Frontend، ابتدا وارد پوشه Frontend شوید:

```
cd frontend
```

سپس دستور زیر را اجرا کنید:

```
mvn javafx:run
```

اگر همه تنظیمات درست باشد، پنجره اصلی برنامه JavaFX باز می‌شود و کاربر می‌تواند از طریق رابط گرافیکی با سامانه کار کند.

اتصال Frontend به Backend

پیش از اجرای کامل Frontend، باید مطمئن شوید که Backend پروژه اجرا شده و روی آدرس مشخصی در دسترس است. در حالت پیش فرض، Backend روی آدرس زیر اجرا می شود:

`http://localhost:8080`

Frontend باید درخواست های خود را به API های Backend ارسال کند. برای مثال، اگر Backend یک API برای دریافت لیست آگهی ها داشته باشد، Frontend می تواند به آدرس زیر درخواست ارسال کند:

`http://localhost:8080/api/ads`

برای ارتباط با Backend در JavaFX، می توان از کلاس `HttpClient` در Java استفاده کرد. بهتر است آدرس اصلی Backend در یک کلاس جداگانه نگهداری شود تا در صورت تغییر پورت یا آدرس، نیاز به تغییر در بخش های مختلف کد نباشد. نمونه ساختار یک کلاس تنظیمات ساده:

```
public class ApiConfig {  
    public static final String BASE_URL = "http://localhost:8080";  
}
```

سپس در بخش های مختلف برنامه می توان از این مقدار برای ساخت آدرس API ها استفاده کرد.

نمونه اجرای اولیه JavaFX

برای بررسی اجرای درست JavaFX، می توان یک برنامه ساده ساخت که یک پنجره با یک متن نمایش دهد:

```
public class MainApplication extends Application {  
  
    @Override  
    public void start(Stage stage) {  
        Label label = new Label("Second-hand JavaFX Frontend is running");  
        Scene scene = new Scene(label, 500, 300);  
  
        stage.setTitle("Second-hand Project");  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```

```

    }

    public static void main(String[] args) {
        launch();
    }
}

```

اگر با اجرای دستور `mvn javafx:run` پنجره‌ای شامل این پیام نمایش داده شود، یعنی JavaFX به درستی روی سیستم اجرا شده است.

📌 نکات مهم در طراحی Frontend

رابط کاربری باید ساده، قابل فهم و منظم باشد. هدف اصلی این پروژه طراحی یک رابط بسیار حرفه‌ای یا شبیه‌سازی یک پلتفرم خاص نیست، بلکه پیاده‌سازی درست منطق برنامه و اتصال مناسب Frontend به Backend اهمیت بیشتری دارد.

با این حال، رعایت موارد زیر الزامی است:

- هر صفحه باید وظیفه مشخصی داشته باشد.
- از قرار دادن کل منطق برنامه داخل یک کلاس خودداری شود.
- ارتباط با Backend بهتر است در کلاس‌های جداگانه Service انجام شود.
- خطاهای دریافتی از Backend باید به شکل مناسب به کاربر نمایش داده شوند.
- ورودی‌های کاربر باید قبل از ارسال به Backend بررسی شوند.
- طراحی برنامه باید به گونه‌ای باشد که کاربر بتواند بین صفحه‌ها جابه‌جا شود.

📌 خطاهای رایج هنگام اجرای Frontend

در هنگام اجرای Frontend ممکن است خطاهای مختلفی رخ دهد. چند مورد از خطاهای رایج عبارت‌اند از:

- نصب نبودن Java یا تنظیم نبودن مسیر آن
- نصب نبودن Maven
- اضافه نشدن وابستگی‌های JavaFX در فایل `pom.xml`
- اشتباه بودن نام کلاس اصلی در پلاگین JavaFX
- اجرا نبودن Backend هنگام ارسال درخواست
- اشتباه بودن آدرس یا پورت Backend

● ناسازگاری نسخه JavaFX با نسخه Java نصب شده

در صورتی که Frontend اجرا شود اما نتواند اطلاعات را از Backend دریافت کند، ابتدا باید بررسی شود که Backend در حال اجرا باشد و آدرس API ها به درستی در Frontend تنظیم شده باشند.

خروجی مورد انتظار این مرحله

در پایان این مرحله، دانشجو باید بتواند برنامه JavaFX را اجرا کند و حداقل یک پنجره گرافیکی مشاهده نماید. همچنین Frontend باید بتواند یک درخواست ساده به Backend ارسال کند و پاسخ آن را دریافت و در رابط کاربری نمایش دهد.

راهنمای راه اندازی Repository در GitHub

در این پروژه، دانشجویان باید کد خود را در یک Repository روی GitHub نگهداری کنند. استفاده از Git باعث می شود روند توسعه پروژه قابل پیگیری باشد و تغییرات پروژه در طول زمان مشخص بماند. همچنین استفاده از GitHub باعث می شود تحویل پروژه، بررسی کدها و مشاهده روند پیشرفت اعضای گروه ساده تر شود.

ساخت Repository در GitHub

برای شروع، یکی از اعضای گروه باید وارد حساب GitHub خود شود و یک Repository جدید برای پروژه ایجاد کند. نام Repository بهتر است واضح و مرتبط با پروژه باشد. برای مثال:

```
secondhand-java-project  
advanced-programming-secondhand  
javafx-spring-secondhand
```

هنگام ساخت Repository، بهتر است گزینه های زیر در نظر گرفته شوند:

● انتخاب نام مناسب برای Repository

● اضافه کردن فایل README

● انتخاب فایل .gitignore مناسب برای Java و Maven

ساختار پیشنهادی Repository

ساختار Repository باید منظم باشد و Backend و Frontend در پوشه‌های جداگانه قرار بگیرند. ساختار پیشنهادی به صورت زیر است:

```
secondhand-java-project/
|-- backend/
|   |-- src/
|   |-- pom.xml
|   `-- README.md
|
|-- frontend/
|   |-- src/
|   |-- pom.xml
|   `-- README.md
|
|-- docs/
|   |-- report.pdf
|   `-- screenshots/
|
|-- README.md
|-- .gitignore
`-- docker-compose.yml
```

پوشه backend شامل کدهای Spring Boot است. پوشه frontend شامل کدهای JavaFX است. پوشه docs می‌تواند شامل گزارش، تصاویر خروجی و مستندات پروژه باشد.

وجود فایل docker-compose.yml الزامی نیست، اما اگر گروه برای راه‌اندازی PostgreSQL از Docker Compose استفاده کند، می‌تواند این فایل را نیز در Repository قرار دهد.

راه‌اندازی Git در سیستم

اگر پروژه ابتدا روی سیستم یکی از اعضای گروه ساخته شده باشد، برای اتصال آن به GitHub می‌توان از دستورهای زیر استفاده کرد.

ابتدا وارد پوشه اصلی پروژه شوید:

```
cd secondhand-java-project
```

سپس Git را در پروژه فعال کنید:

```
git init
```

فایل‌ها را به مرحله آماده‌سازی اضافه کنید:

```
git add .
```

اولین commit را ثبت کنید:

```
git commit -m "Initial project structure"
```

سپس Repository ساخته‌شده در GitHub را به پروژه متصل کنید:

```
git remote add origin https://github.com/username/secondhand-java-project.git
```

ن. دستور، به جای username باید نام کاربری GitHub صاحب Repository و به جای secondhand-java-project نام Repository واقعی قرار گیرد.

در نهایت، کدها را روی GitHub ارسال کنید:

```
git branch -M main
```

```
git push -u origin main
```

دریافت پروژه از GitHub

اگر یکی از اعضای گروه بخواهد پروژه را روی یک سیستم دیگر دریافت کند، می‌تواند از دستور زیر استفاده کند:

```
git clone https://github.com/username/secondhand-java-project.git
```

پس از اجرای این دستور، یک پوشه جدید با نام پروژه ساخته می‌شود و فایل‌های Repository داخل آن قرار می‌گیرند.

استفاده از commit در طول پروژه

اعضای گروه نباید فقط در پایان پروژه یک commit انجام دهند. لازم است روند پیشرفت پروژه در commit های مختلف مشخص باشد. هر commit باید یک تغییر مشخص و قابل فهم را نشان دهد.

نمونه پیام های مناسب برای commit:

```
Add user entity and repository
Implement user registration API
Create JavaFX login page
Add advertisement search filters
Implement chat message service
Fix rating validation bug
Update README with setup instructions
```

پیام هایی مانند موارد زیر مناسب نیستند:

```
update
final
fix
new
project
```

هر commit باید نشان دهد چه تغییری در پروژه انجام شده است.

فایل .gitignore

در Repository باید فایل .gitignore وجود داشته باشد تا فایل های اضافی، خروجی های build و فایل های حساس وارد GitHub نشوند.

اگر گروه از SQLite استفاده می کند و لازم است فایل پایگاه داده برای تست پروژه تحویل داده شود، باید طبق دستور استاد عمل کند. در حالت عادی، بهتر است فایل های پایگاه داده و فایل های محلی در GitHub قرار نگیرند و روش ساخت یا اجرای آن ها در README توضیح داده شود.

فایل README

وجود فایل README.md در ریشه Repository الزامی است. این فایل باید توضیح دهد پروژه چیست و چگونه اجرا می شود.

مواردی که باید در README نوشته شوند:

- عنوان پروژه
- نام اعضای گروه
- توضیح کوتاه درباره پروژه
- تکنولوژی‌های استفاده‌شده
- پیش‌نیازهای اجرا
- نحوه اجرای Backend
- نحوه اجرای Frontend
- نحوه راه‌اندازی روش ذخیره‌سازی، مانند Database یا فایل
- اطلاعات حساب‌های تست، مانند کاربر عادی و مدیر
- توضیح امکانات پیاده‌سازی‌شده

📌 نحوه کار گروهی با Git

این پروژه باید به صورت گروه‌های دو نفره انجام شود. هر دو عضو گروه باید در Repository مشارکت داشته باشند. بهتر است هر عضو برای کار خود branch جداگانه بسازد و پس از کامل شدن تغییرات، آن را با branch اصلی ادغام کند.

ساخت branch جدید:

```
git checkout -b feature/login-page
```

ثبت تغییرات:

```
git add .
git commit -m "Implement login page"
```

ارسال branch به GitHub:

```
git push origin feature/login-page
```

در این پروژه دو نفره، بهتر است هر عضو به اندازه سهم خود commit داشته باشد. نبودن commit از طرف یک عضو می‌تواند نشان‌دهنده مشارکت کم یا نامشخص بودن سهم او در پروژه باشد.

مواردی که نباید در GitHub قرار گیرند

دانشجویان نباید فایل‌های زیر را در Repository قرار دهند:

- فایل‌های خروجی build مانند پوشه target
- فایل‌های تنظیمات شخصی IDE
- رمز عبورهای واقعی
- فایل‌های حجیم غیرضروری
- فایل‌های log
- فایل‌های موقت سیستم‌عامل
- اطلاعات حساس مربوط به Database

در صورتی که پروژه به فایل‌های نمونه یا داده‌های اولیه نیاز دارد، بهتر است فایل نمونه بدون اطلاعات حساس قرار داده شود و نحوه استفاده از آن در README توضیح داده شود.

تحويل Repository

در زمان تحويل، گروه باید لینک Repository را به صورت آنلاین ارسال کند. Repository باید شامل سورس کامل پروژه، README، فایل‌های لازم برای اجرا و مستندات مورد نیاز باشد.

در صورتی که Repository private باشد، اعضای گروه باید دسترسی لازم را برای استاد یا دستیار آموزشی فعال کنند. اگر دسترسی داده نشود، پروژه قابل بررسی نخواهد بود.

پیش از تحويل نهایی، اعضای گروه باید بررسی کنند که پروژه از روی Repository قابل اجرا باشد. یعنی اگر ارزیاب پروژه را clone کند، بتواند با استفاده از توضیحات README آن را اجرا کند.